

PROMETEUS

Sistema de archivos virtual
en memoria

Integrantes:

Gamarra Cure Jeferson Fabian

Cuello Inche Manuel Benito

Luna Roca Fabrizio Santiago

Flores Zambora Airthon Daniel

Docente: Juan Luis Herencia Guerra



USO DE PROMETEUS

¿QUÉ PROBLEMA RESUELVE?

Beneficios del Sistema de Archivos en Memoria



APLICACIONES PRÁCTICAS

Dispositivos Embebidos

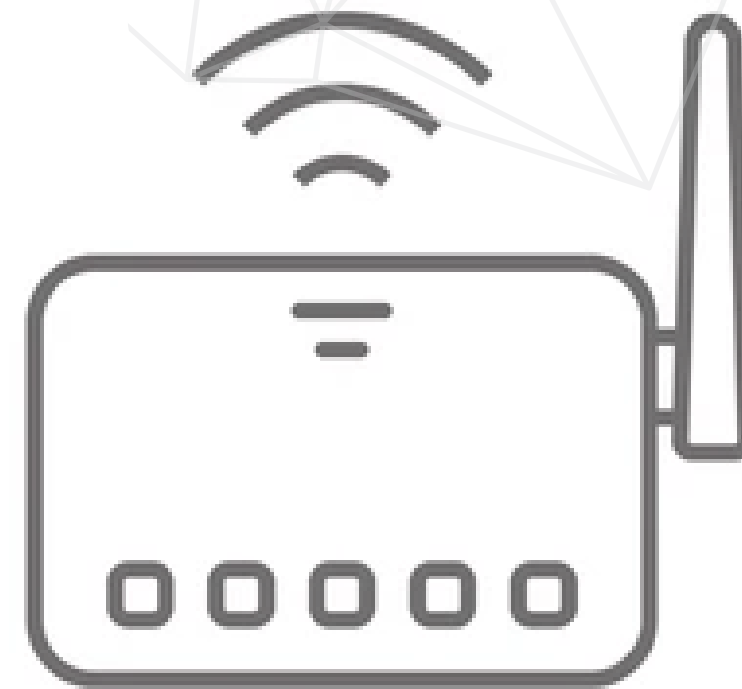
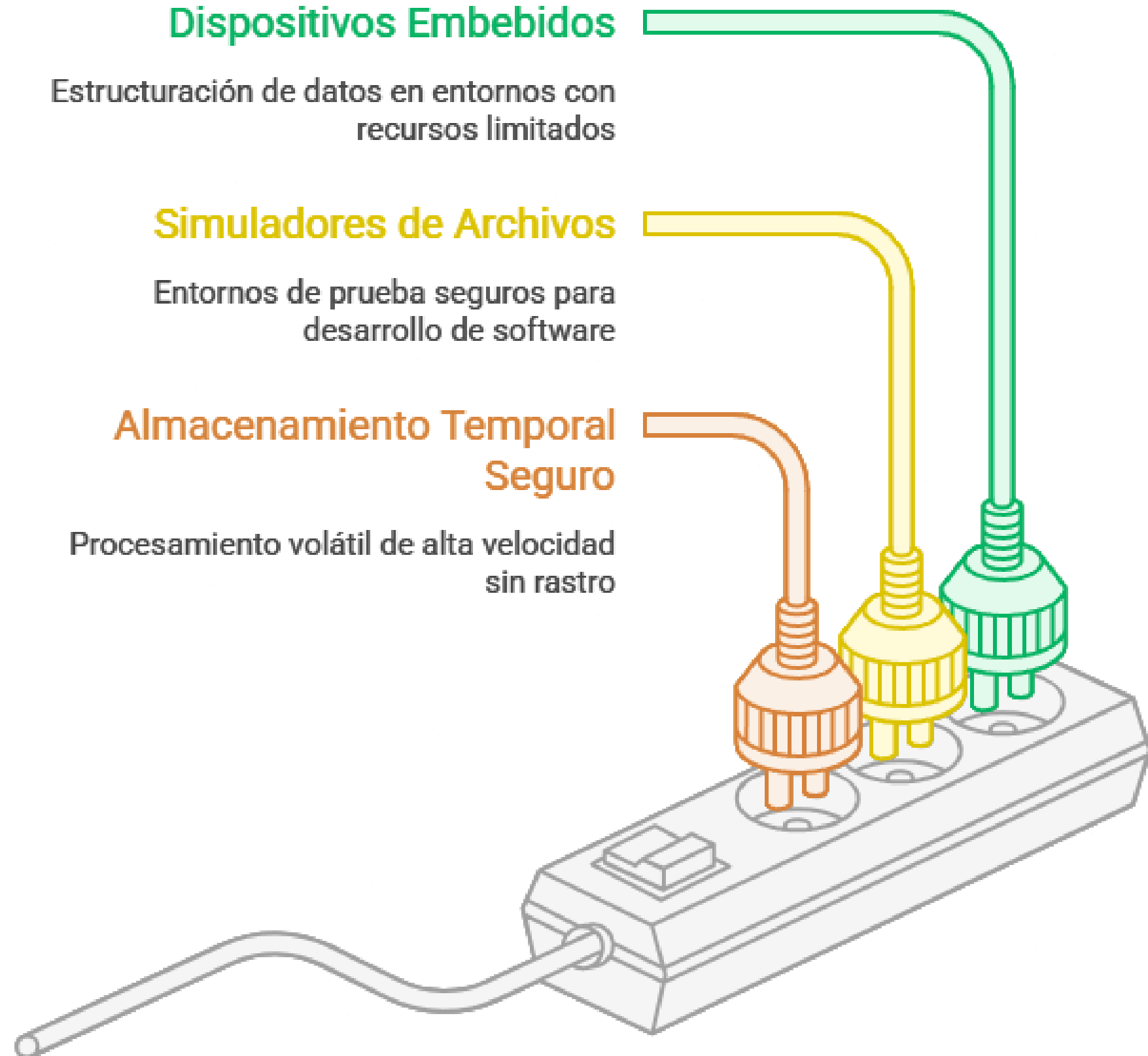
Estructuración de datos en entornos con recursos limitados

Simuladores de Archivos

Entornos de prueba seguros para desarrollo de software

Almacenamiento Temporal Seguro

Procesamiento volátil de alta velocidad sin rastro



ROUTER

XV6 UNIX

xv6

xv6: A Simple,
Unix-like Teaching
Operating System

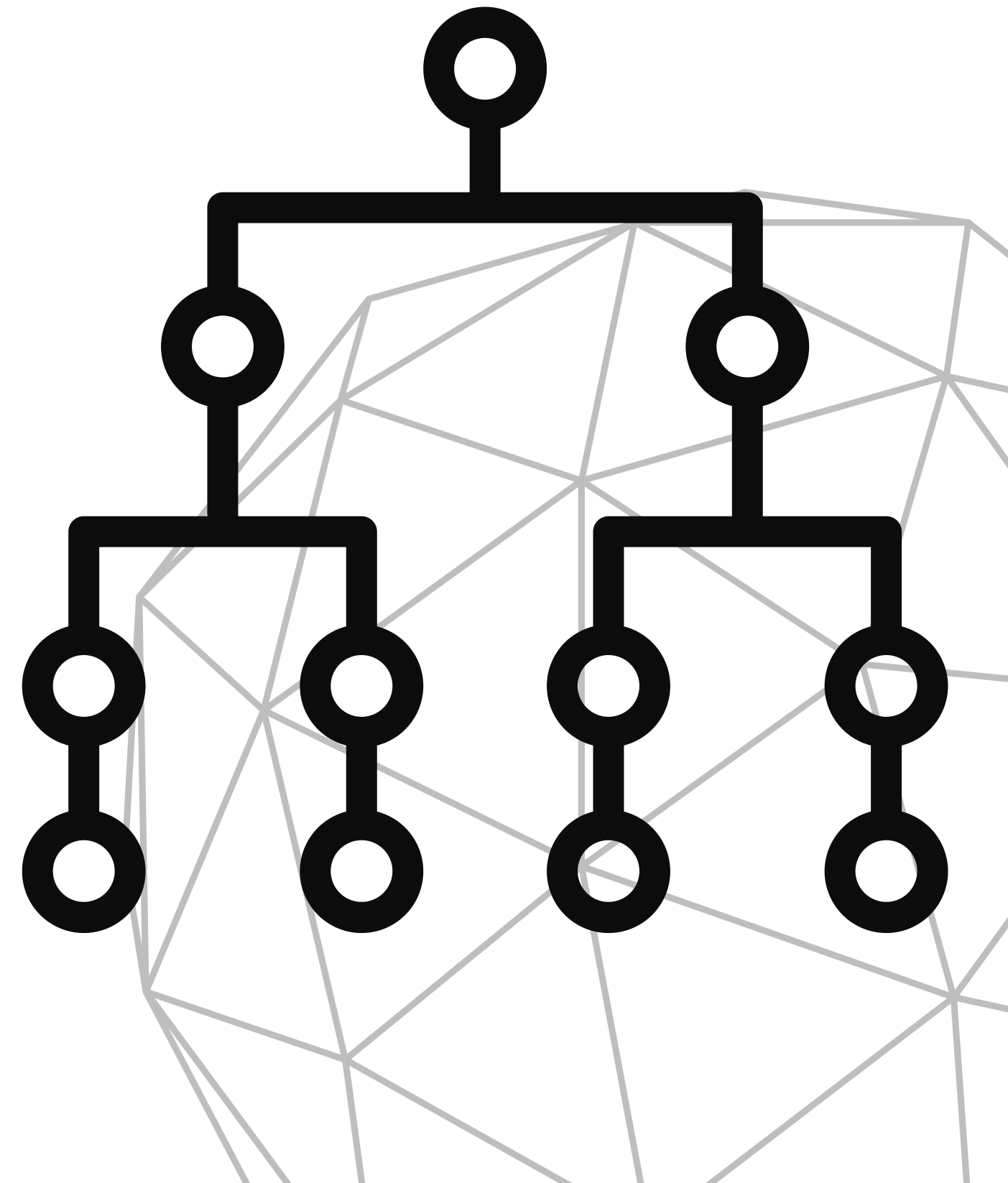
*Russ Cox, Frans Kaashoek,
Robert Morris*

ARBOL MULTIPLE (ARBOL B)

Un árbol múltiple es una generalización de un árbol binario.

Mientras que un nodo en un árbol binario solo puede tener como máximo 2 hijos y 1 clave (valor), un nodo en un árbol multivía de orden M puede tener hasta M hijos y $M-1$ claves.

Un *árbol B* solo guarda información en las hojas del árbol mientras que los nodos internos solo almacenan direcciones hacia otros nodos





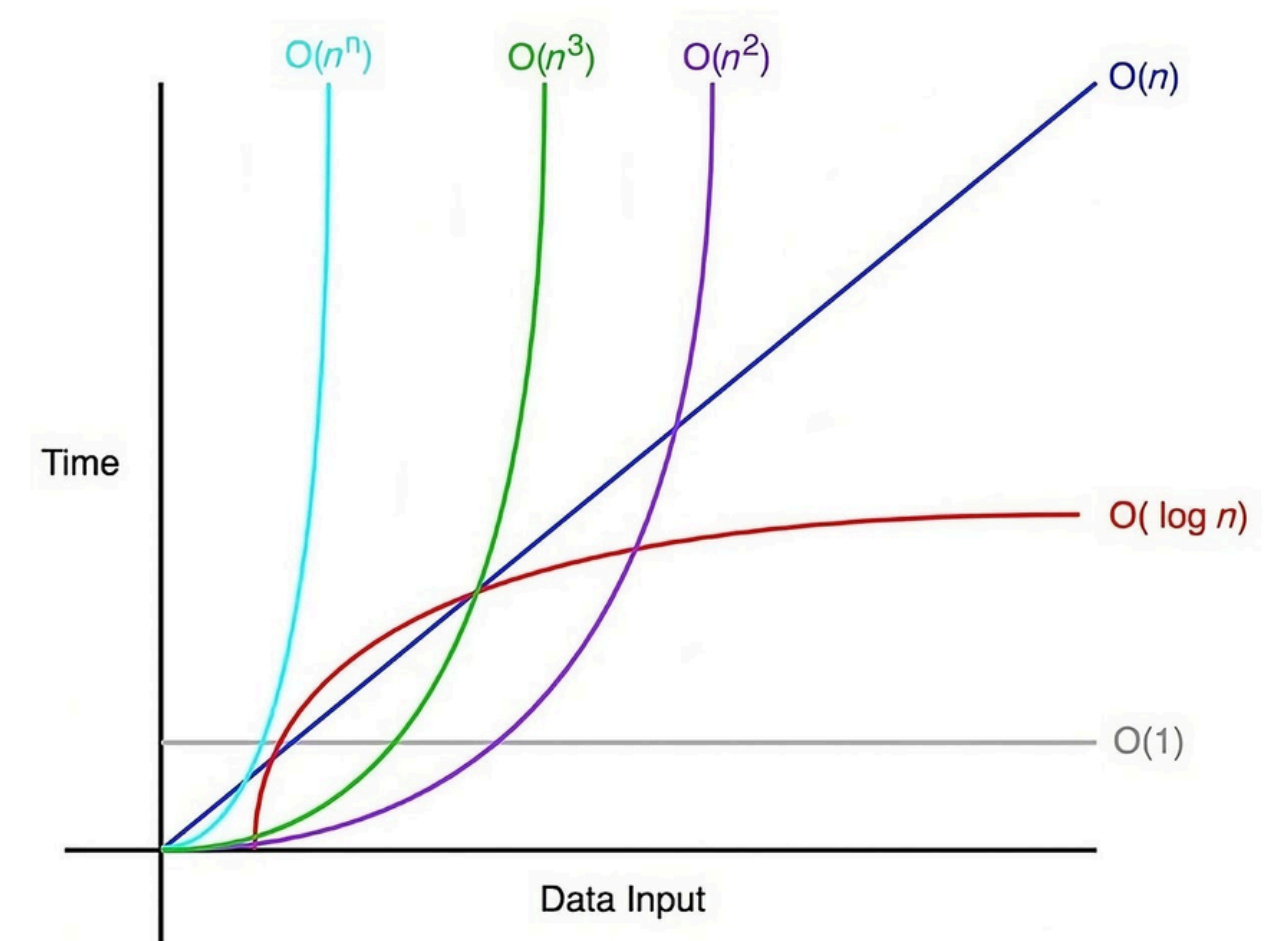
¿PORQUE SU USO?

A comparación con un árbol binario, una mayor ramificación permite una altura pequeña y facilita que un nodo guarde mas direcciones

COMPLEJIDAD

Los algoritmos de búsqueda, inserción y eliminación son de complejidad $O(\log(n))$, quiere decir que son muy efectivos

No almacena datos pesados dentro de nodos internos, solo en las hojas, por lo que comandos como `ls` son muy rápidos.



ARQUITECTURA DEL ARBOL

FRAGMENTO...

```
typedef struct Nodo {  
    char nombre[MAX_NOMBRE];  
    TipoNodo tipo;  
    struct Nodo *padre;  
    struct Nodo **hijos;  
    int numHijos;  
    char *contenido;  
    int size;  
} Nodo;
```

```
Nodo *crearNodo(const char *nombre, TipoNodo tipo) {  
    Nodo *n = (Nodo *)malloc(sizeof(Nodo));  
    if (!n) { perror("malloc nodo"); exit(1); }  
    strncpy(n->nombre, nombre, MAX_NOMBRE - 1);  
    n->nombre[MAX_NOMBRE - 1] = '\0';  
    n->tipo = tipo;  
    n->padre = NULL;  
    n->hijos = NULL;  
    n->numHijos = 0;  
    n->contenido = NULL;  
    n->size = 0;  
    return n;  
}
```

The background features three decorative geometric elements: a blue wireframe grid in the top left, a grey wireframe sphere in the top right, and a grey wireframe cube in the bottom left.

MANEJO DE MEMORIA Y SERIALIZACIÓN

ASIGNACION DINAMICA

¿Que es?

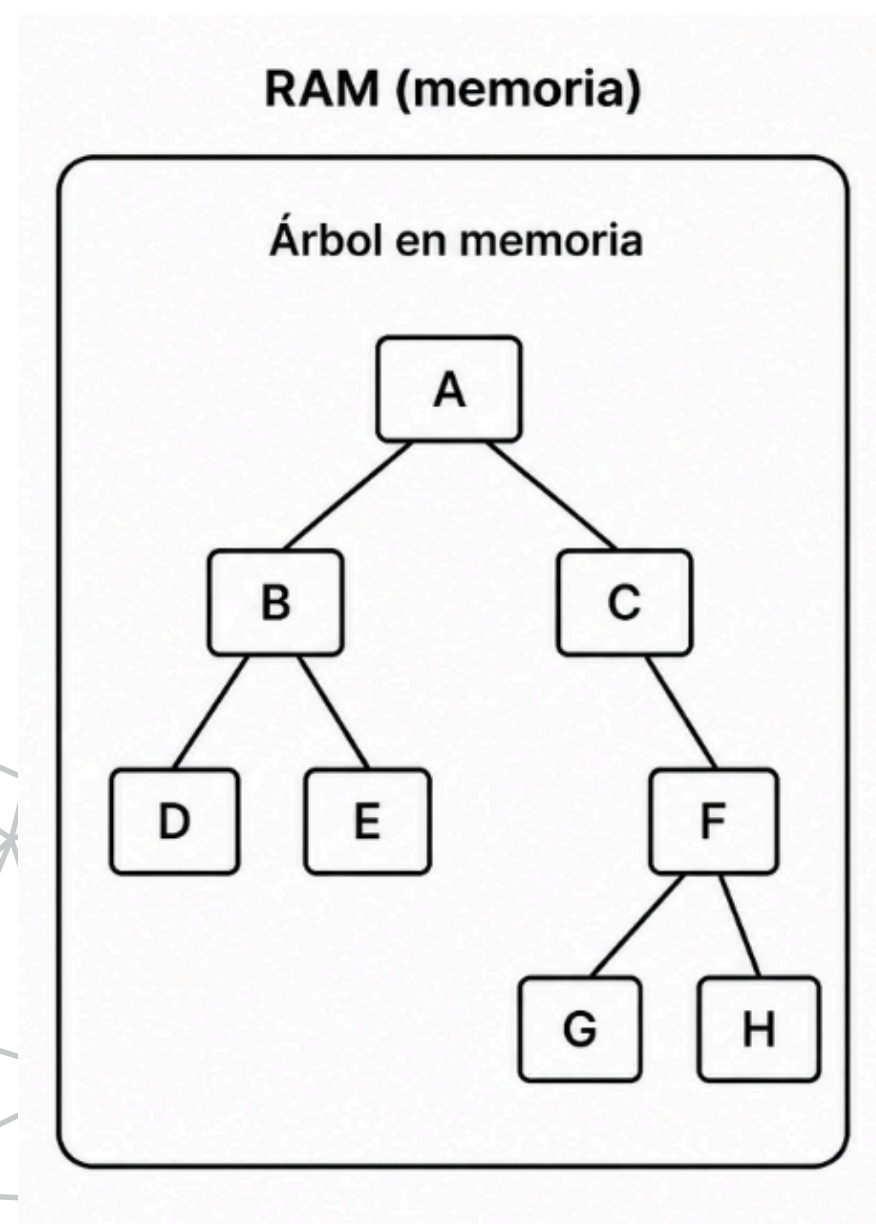
- Usando Malloc

```
Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));  
nuevo->clave = nombre_archivo;
```

al finalizar

- Liberar memoria

```
void free_tree(Nodo *raiz) {  
    if (!raiz) return;  
    free_tree(raiz->hijo_izq);  
    free_tree(raiz->hijo_der);  
    free(raiz);  
}
```



SERIALIZACION BINARIA

De arbol $\longrightarrow$ Byte en binario

```
typedef struct {  
    char nombre[256];  
    int es_directorio;  
    int tamaño;  
} NodoSerializable;
```

Funciones propuestas:

- guardar_arbol()
- cargar_arbol()

Uso de funciones adicionales:

- Fwrite
- Fread



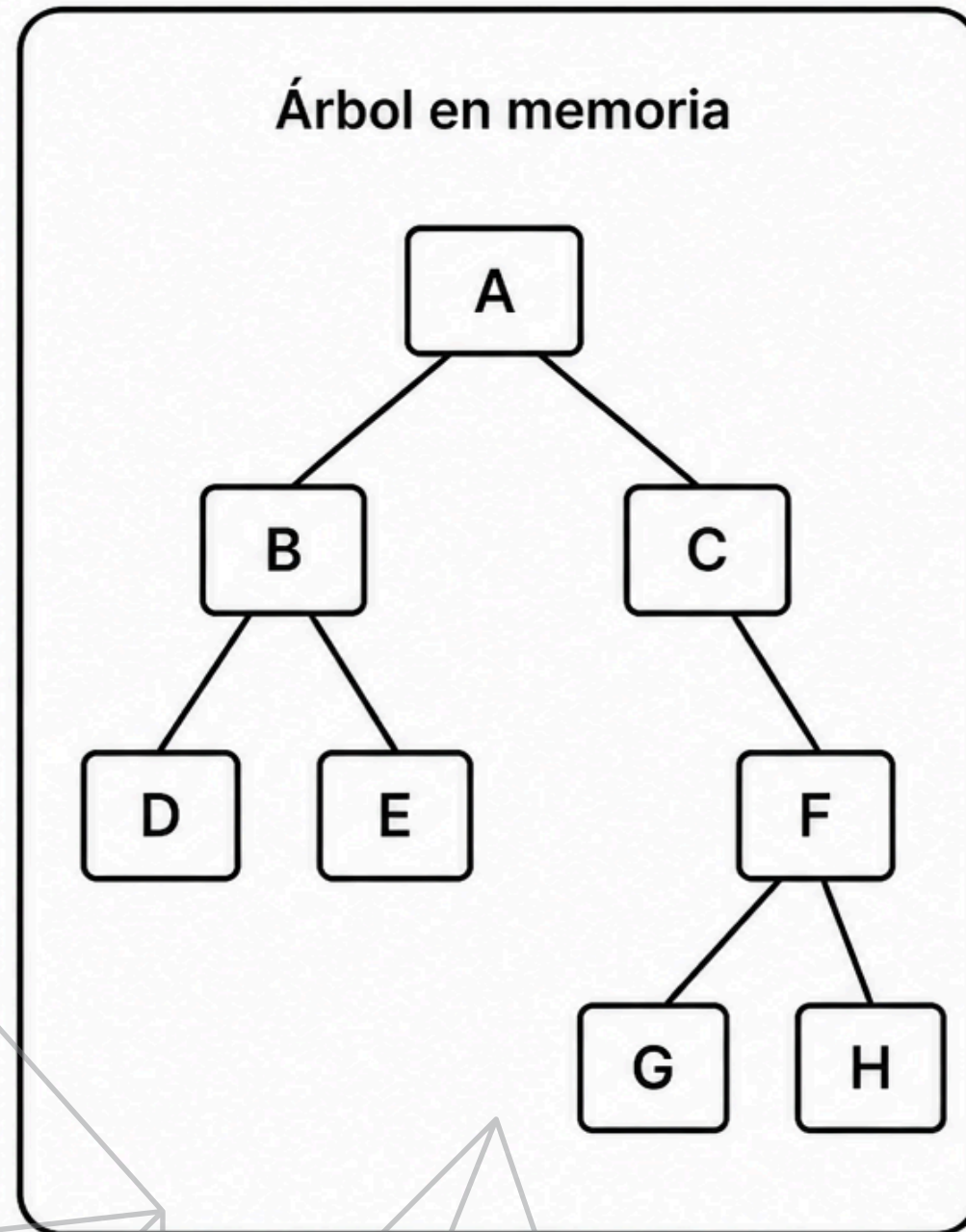
¿POR QUÉ BINARIO?

Característica	Binario	Texto (JSON/XML)
Velocidad	Muy rápida (sin parsing)	Lenta (hay que analizar sintaxis)
Tamaño	Compacto (bytes puros)	Grande (etiquetas, comillas, saltos)
Parsing	No requiere (lo lee directamente la estructura en memoria)	Sí requiere
Legibilidad	No legible por humanos	Legible
Precisión	Preserva floats y enteros exactamente	Puede haber pérdida por conversión

FUTURA IMPLEMENTACION

RAM (memoria)

Árbol en memoria



guardar_arbol()



Disco duro

Archivo binario
(árbol.bin)

0x41	0x02	0x42
0x44	0x45	0x43
0x01	0x46	0x00
0x47	0x48	
...		

cargar_arbol()



IMPLEMENTACIÓN DEL CÓDIGO FUENTE



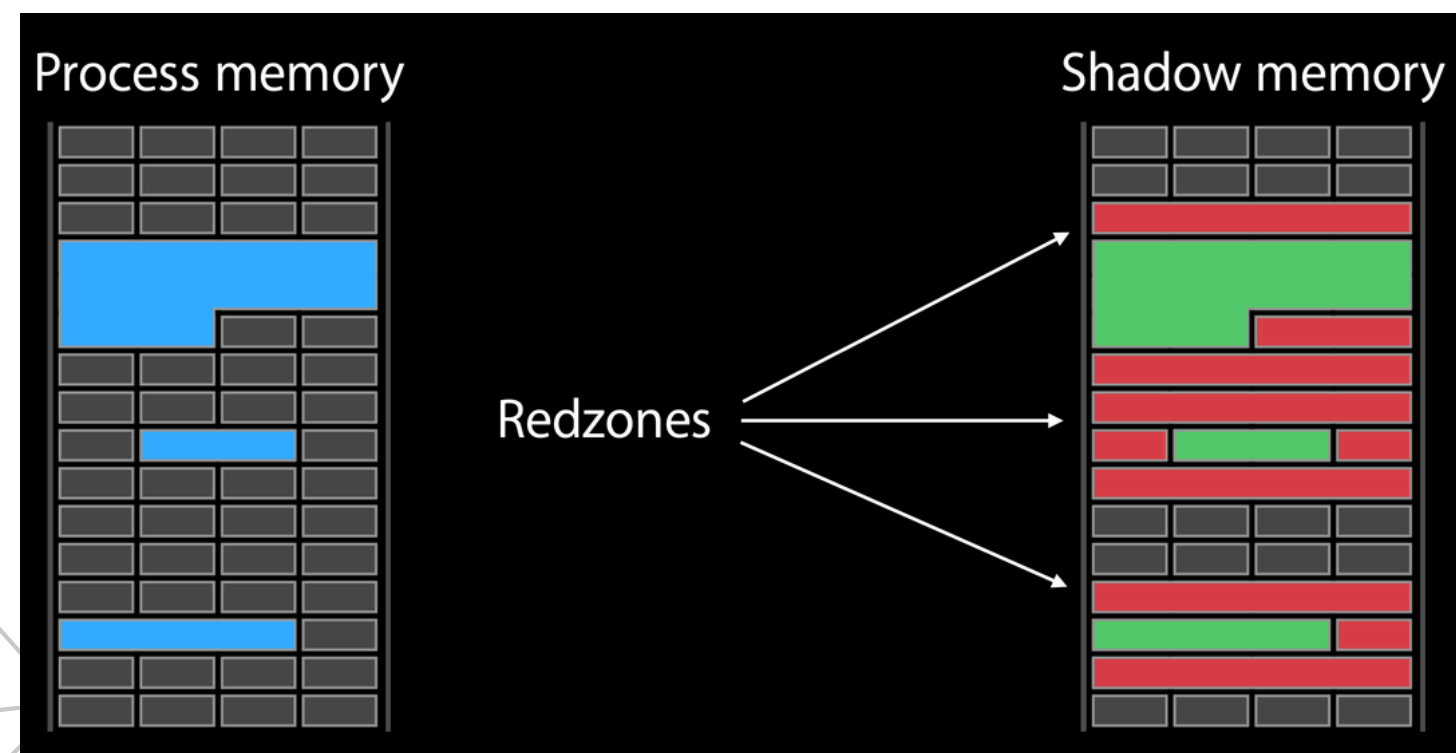
DETECCION ERRORES DE MEMORIA

AddressSanitizer

Herramienta de depuración de código, detecta errores en la memoria dinámica desarrollada por Google

Valgrind memcheck

Es un framework de análisis de programas, la herramienta memcheck monitorea el programa durante su ejecución



Detectan errores de tipo:

- memory leaks
- invalid reads
- memoria no inicializada
- errores con malloc/free
- double free



GRACIAS

